



UNIVERSIDAD
Blas Pascal

PROYECTO FINAL 2026 - TC

Asignatura: Técnicas de Compilación

Fecha: 12/6/2026

Autor:

Tomás Calzada
Octavio Douglas
Ignacio Galvan

Profesor: Francisco Ameri

RESUMEN

Este informe presenta el desarrollo de un compilador educativo para un lenguaje reducido inspirado en C++. El proyecto fue realizado en Java con ANTLR4 y permite reconocer el código fuente, verificar su estructura y detectar errores antes de generar una representación intermedia.

Además del análisis léxico y sintáctico, se incorporaron una tabla de símbolos, controles semánticos, generación de código de tres direcciones y distintas técnicas de optimización. La versión entregada compila correctamente y sus 17 pruebas automatizadas finalizan sin fallos.

1. INTRODUCCIÓN

El trabajo surge como una aplicación práctica de los temas vistos en Técnicas de Compilación. En lugar de abarcar todo C++, se definió un subconjunto manejable que conserva elementos importantes como variables, expresiones, condicionales, ciclos, arreglos básicos y funciones.

El objetivo principal fue construir un proceso de compilación dividido en etapas claras. De esta manera, cada componente tiene una responsabilidad concreta y los errores pueden informarse con la línea y la columna donde ocurren.

2. DESARROLLO

2.1 Gramática y análisis del código

La gramática se encuentra en el archivo MiLenguaje.g4. Allí se definen los tokens y las reglas válidas del lenguaje. ANTLR4 utiliza esta información para generar el lexer y el parser. El primero separa el texto en tokens y el segundo comprueba que esos tokens respeten una estructura válida.

El lenguaje acepta tipos como int, float, double, char, string y bool. También reconoce asignaciones, operaciones aritméticas y lógicas, if-else, while, for, break, continue, funciones, return y cout. Cuando la sintaxis es correcta, el programa muestra el árbol de parseo generado por ANTLR.

2.2 Análisis semántico

Luego del análisis sintáctico se ejecuta un visitor semántico. Esta etapa controla que las variables hayan sido declaradas, evita declaraciones duplicadas y valida el uso de funciones, parámetros, return, break y continue. Para administrar los nombres se utiliza una tabla de símbolos con ámbitos globales y locales.

2.3 Código intermedio y optimización

Si no hay errores, el compilador genera código de tres direcciones. Las expresiones se dividen en instrucciones simples mediante temporales, mientras que los condicionales y ciclos se representan con etiquetas y saltos. El resultado se guarda en un archivo de texto.

El proyecto incluye optimización de código muerto, propagación de constantes, simplificación de expresiones y eliminación de sentencias redundantes. En la configuración

actual se aplica una técnica por ejecución, aunque la estructura permite incorporar un proceso combinado más adelante.

2.4 Pruebas y resultados

Se realizaron pruebas sobre la tabla de símbolos, el análisis semántico y la salida de mensajes. También se utilizaron programas de ejemplo válidos y archivos con errores intencionales para comprobar los diagnósticos. El 12 de junio de 2026 se ejecutó Maven y se obtuvieron 17 pruebas aprobadas, sin fallos ni errores.

Como resultado, el sistema procesa el archivo fuente, muestra los tokens, construye el árbol, valida los símbolos y genera archivos con el código intermedio y optimizado. Los mensajes de éxito, advertencia y error se diferencian por color para facilitar la lectura en consola.

3. CONCLUSIONES

El proyecto permitió integrar en una sola aplicación las etapas principales de un compilador. ANTLR4 simplificó la construcción del análisis léxico y sintáctico, mientras que el uso de visitors ayudó a separar la validación semántica y la generación de código.

La solución cumple con el alcance propuesto y sirve para observar de forma concreta cómo se transforma un programa fuente. Como mejoras futuras quedan una validación de tipos más completa, más pruebas para los optimizadores y la posibilidad de aplicar varias optimizaciones en una misma ejecución.

4. BIBLIOGRAFÍA

Aho, A. V., Lam, M. S., Sethi, R. y Ullman, J. D. (2007). Compilers: Principles, Techniques, and Tools. Pearson.

Parr, T. (2013). The Definitive ANTLR 4 Reference. Pragmatic Bookshelf.

ANTLR. Documentación oficial. <https://www.antlr.org/>

Oracle. Documentación de Java. <https://docs.oracle.com/en/java/>

5. FUENTES DE DATOS

La información técnica se obtuvo directamente del repositorio del proyecto y de sus archivos fuente: MiLenguaje.g4, App.java, los visitors, la tabla de símbolos, los optimizadores y las pruebas unitarias.

Repositorio:

https://github.com/Ignagalvan/PROYECTO_FINAL_TC_CALZADA_DOUGLAS_GALVAN

Verificación local: ejecución de mvn test realizada el 12 de junio de 2026. Resultado: 17 pruebas, 0 fallos y 0 errores.

6. GLOSARIO

ANTLR4: herramienta que genera analizadores a partir de una gramática.

Lexer: componente que convierte los caracteres del archivo en tokens.

Parser: componente que verifica la estructura sintáctica del programa.

Token: unidad reconocida, por ejemplo una palabra reservada o un número.

Árbol de parseo: representación jerárquica de las reglas reconocidas.

Tabla de símbolos: estructura que guarda variables, funciones y ámbitos.

Código intermedio: representación simple previa al código final.

Optimización: transformación que mejora el código sin cambiar su resultado.